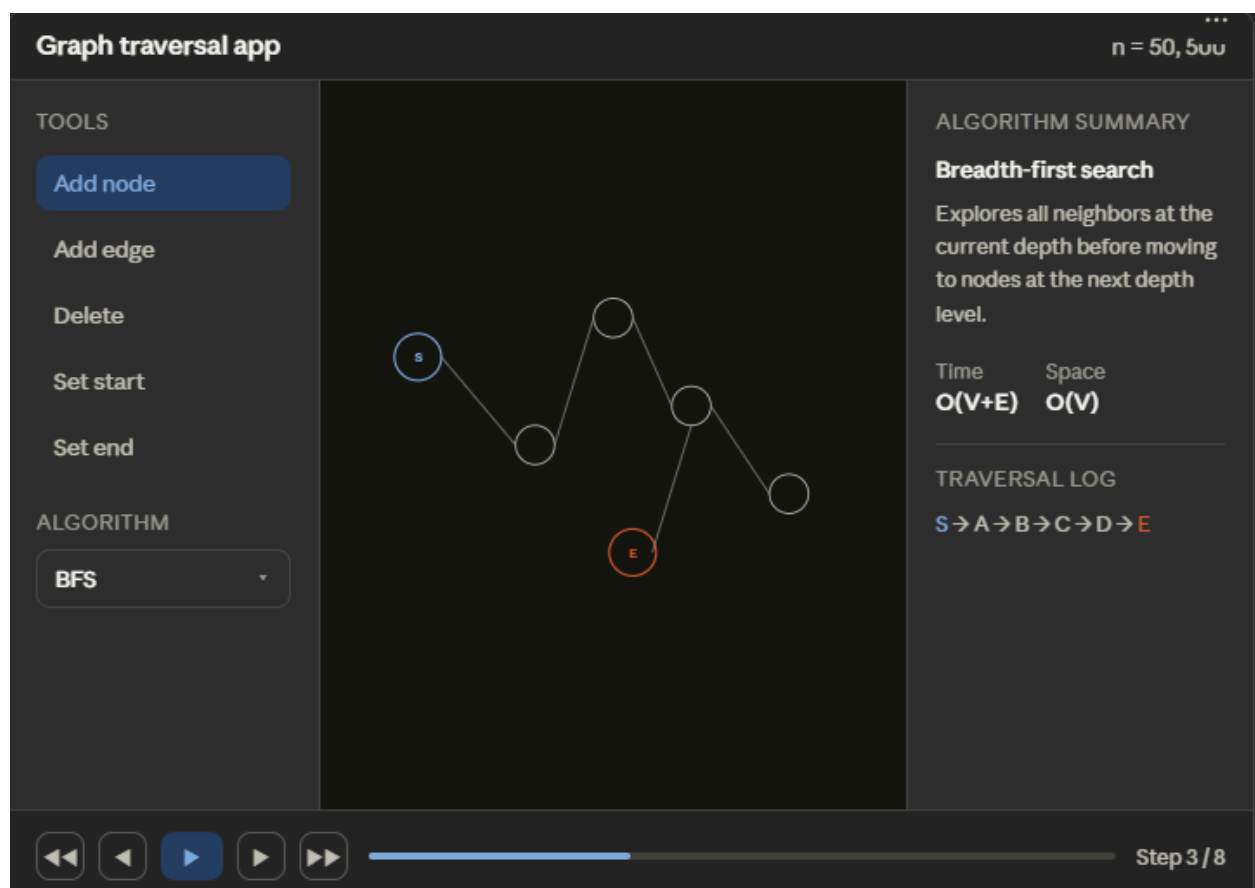


## Graph Traversal Project

### Overview:

The intention of this project is to create a user-friendly web application that depicts various traversing algorithms on a given graph. Upon access, users are presented with a series of traversal algorithms that, once selected, populates an area of the user's device with a graph satisfying the initial constraints of the chosen algorithm. The produced graph can be modified via adding additional nodes or edges to the graph, with a visual indicator informing if the modifications made conform to the constraints of the chosen algorithm. Once all conditions are satisfied, the user may visualize the selected algorithm either by playing through continuously or stepping through the process.



Wireframe generated in Claude AI – created via referencing a handdrawn wireframe by me.

### Architecture

The project consists of four major modules: graph data module, graph editor/UI module, algorithm module, and playback/visualization module. These modules create clear separations of responsibilities, reducing development complexity, aiding in testability, while providing a clear roadmap for creating the application. Data flows in one direction: user interactions modify the

graph data module, whose state feeds the algorithm engine, which provides output to the playback/visualization module.

#### Module one - graph data model:

The graph data model is the single source of truth within the program. It's tasked with maintaining the internal structure of the currently populated graph. Structurally, the relationship between nodes and its neighbors are maintained by an adjacency list, which is implemented using a map, where the keys are nodes and neighbors are corresponding value pairs. Additional information relevant for other modules are stored within the node, as they are implemented as objects. For writing operations, the module supports adding and removing nodes and edges, with additional support for updating nodes. Reading is most relevant for the algorithm engine, which is achieved through an interface that provides a node's neighbors.

#### Module two - graph editor/UI

The module handles two functions of the web application. First, it provides the operations that enable users to interact with the graph. Interactions include adding, removing, and repositioning nodes or edges. Simultaneously, the module ensures modifications made by the user are communicated to the graph data model. In addition to providing interactivity and syncing, the module handles the rendering and placing of SVGs on the canvas of the application.

#### Module three - algorithm engine

The algorithm engine stores and executes the traversal algorithms that run on the graph provided by the graph data module. Information pertaining to which algorithm to run is processed by the graph editor/UI module, which is provided to the algorithm engine. Once a traversal algorithm and starting node is selected, the algorithm engine builds an ordered array of objects while the algorithm executes. The properties of the object stores information about the current node, from node, and the state of the data structure when that object was created. After execution, the resulting array is supplied to the playback engine.

#### Module four - playback visualization engine

The playback/visualization engine is responsible for converting the data created by the algorithm engine into an accurate visual representation for the user. In doing so, the engine must be able to handle the following cases: continuous playback and step-through playback. The case of continuous playthrough is triggered when the user clicks the play button. In this scenario, a loop runs until either execution terminates (with a delay built in to each iteration for enhanced user experience), indicating the visualization is finished, or the user clicks the pause button, which halts execution on the current loop. If paused, execution continues once the play button is clicked. Alternatively, the user can click the forward button, which triggers the second case. During the step-through scenario, iteration is continued by successive clicks of the forward button, with the backwards button reversing to the previous state of iteration. During

either scenario, the user is informed visually via highlights of the from and to nodes provided by the input array and presented with a visual of the relevant data structure for the chosen traversal.

#### Design decisions

The design of the application required evaluating tradeoffs at several points during planning. Two decisions in particular shaped the architecture: how to render the graph canvas, and the boundary of responsibility between the algorithm engine and the visualization layer.